TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE OAXACA

MATERIA:

LENGUAJES Y AUTÓMATAS II

CLAVE DE MATERIA:

SCD1016

CARRERA:

INGENIERÍA EN SISTEMAS COMPUTACIONALES

PROYECTO INTEGRADOR – FASE 2: DESARROLLO (REPORTE DE AVANCE)

EmoLang

ALUMNOS:

GARCIA GABRIEL BETSAIDA
GINER COACHE NIERIKA
JIMENEZ LOPEZ FERNANDO ARATH
LEÓN RAMÍREZ CARLOS PAVEL
ORTIZ MARTÍNEZ JOSÉ JULIÁN

NOMBRE DEL PROFESOR:

VELAZQUEZ HERNANDEZ MARICARMEN MONTSERRAT

GRUPO: 7SB

OAXACA DE JUÁREZ, OAXACA, OCTUBRE DE 2026

Contenido

1. Introducción . . . 3

2. Herramientas y organización del proyecto . . . 3

3. Especificación del lenguaje (objetivo 1) . . . 4

- 3.1 Estructura de un programa . . . 4
- 3.2 Componentes léxicos . . . 5
- 3.3 Gramática . . . 5
- 3.4 Tipos y reglas de conversión . . . 6
- 3.5 Ámbitos . . . 6
- 3.6 Semántica de ejecución . . . 6

4. Análisis léxico (objetivo 2) . . . 7

5. Análisis sintáctico (objetivo 2) . . . 8

6. Análisis semántico (objetivo 3) . . . 9

7. Validaciones propias del dominio (objetivo 4) . . . 9

8. Generación de autómatas (objetivo 5) . . . 11

- 8.1 Autómata de un gesto . . . 11
- 8.2 Tolerancia a interrupciones . . . 12
- 8.3 Secuencias . . . 13
- 8.4 Máquinas de estado . . . 14

9. Motor de ejecución y sensor de cámara (objetivo 6) . . . 14

- 9.1 Ciclo por cuadro . . . 14
- 9.2 Velocidad de la cámara . . . 15
- 9.3 Métricas faciales . . . 15

10. Módulos de dominio (objetivo 7) . . . 15

11. Pruebas y resultados (objetivo 8) . . . 17

- 11.1 Pruebas unitarias . . . 17
- 11.2 Evaluación con escenarios simulados . . . 18
- 11.3 Robustez ante ruido . . . 19
- 11.4 Pruebas con cámara real . . . 19

12. Cambios respecto al anteproyecto . . . 22

13. Limitaciones y trabajo pendiente . . . 23

14. Puntos para revisión . . . 24

Referencias . . . 24

Anexo A. Programa del módulo de conducción . . . 25

Anexo B. Programa del módulo de rehabilitación . . . 27

Anexo C. Cómo reproducir los resultados . . . 28

1. Introducción

Este documento reporta el avance de la segunda fase del proyecto integrador: el desarrollo de EmoLang, el lenguaje de dominio específico propuesto en el anteproyecto para describir gestos faciales como patrones temporales, validarlos y compilarlos a autómatas finitos que se ejecutan con una cámara convencional. Se describe lo implementado para cada objetivo específico, las pruebas realizadas con sus resultados y los cambios que surgieron respecto a la propuesta original. La sección 14 reúne los puntos que el equipo somete a revisión.

Las ocho partes planteadas en los objetivos específicos tienen una primera versión funcional. El compilador procesa un programa completo y genera sus autómatas, y el motor de ejecución los evalúa en tiempo real con la cámara web. La evaluación con grabaciones de varios participantes (objetivo 8) tiene ya las herramientas y el protocolo, pero todavía no se realiza; los resultados cuantitativos de este reporte provienen de datos simulados y de pruebas con cámara hechas por un integrante del equipo.

Tabla 1. Estado de los objetivos específicos.

Objetivo específico	Estado	Archivos principales
1. Sintaxis y gramática libre de contexto	Implementado	docs/gramatica.md
2. Analizador léxico y sintáctico (AST)	Implementado	lexico.py, sintactico.py, nodos.py
3. Analizador semántico (símbolos, ámbitos, tipos)	Implementado	semantico.py, tipos.py
4. Validaciones propias del dominio	Implementado	validaciones.py
5. Generador de autómatas (tabla y diagrama)	Implementado	automatas.py
6. Motor de ejecución con MediaPipe	Implementado; probado con cámara	motor.py, ejecucion.py, camara.py
7. Módulos de conducción y rehabilitación	Implementado	modulos.py
8. Evaluación (exactitud, FP/FN, latencia)	Simulada: hecha. Con participantes: pendiente	evaluacion.py, captura.py, diagnostico.py

2. Herramientas y organización del proyecto

El proyecto está escrito en Python. El compilador y el motor no tienen dependencias externas y funcionan desde Python 3.10. La parte de visión usa MediaPipe 1.1.1 (Face Landmarker: 478 puntos faciales y 52 coeficientes de expresión, o *blendshapes*) y OpenCV 5.0; como MediaPipe no tiene versiones para Python 3.13 o posterior, esa parte se ejecuta en un entorno virtual con Python 3.11. Las pruebas usan el módulo estándar *unittest*. El código fuente suma unas 3,900 líneas, más 360 de pruebas.

Los analizadores léxico y sintáctico se escribieron a mano, sin generadores como ANTLR o PLY, para que cada fase del compilador quede visible y corresponda directamente con la teoría de la materia. La Figura 1 muestra cómo se conectan las fases.

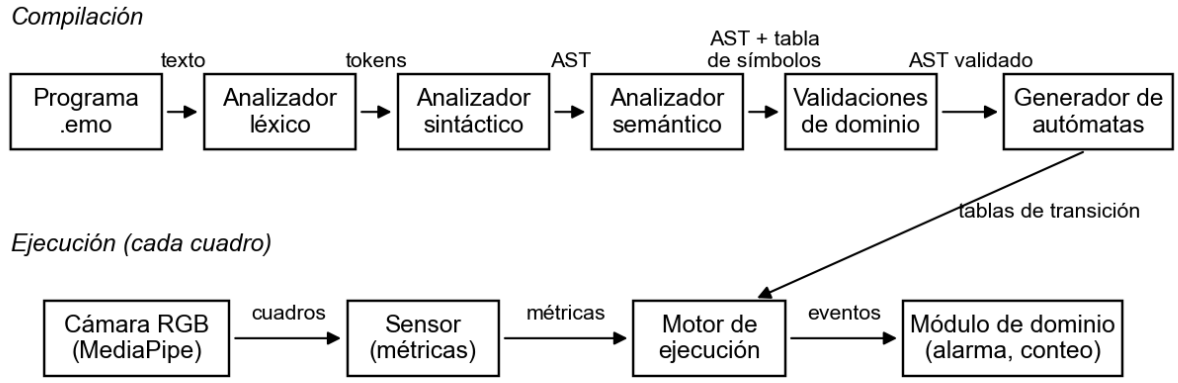


Figura 1. Arquitectura: fases de compilación y ciclo de ejecución.

Todas las funciones se usan desde la línea de comandos:

Tabla 2. Comandos de la herramienta (python -m emolang <comando>).

Comando	Función
lexico, sintactico, semantico	Muestran la salida de cada fase: tokens, árbol sintáctico, tabla de símbolos y diagnósticos.
compilar	Genera los autómatas: tablas de transición, diagramas (Mermaid y Graphviz) y JSON.
ejecutar	Ejecuta el programa con un sensor simulado, un CSV grabado o la cámara.
evaluar	Calcula exactitud, falsos positivos/negativos y latencia contra datos etiquetados.
capturar	Graba métricas de la cámara mientras una persona etiqueta los gestos con el teclado.
diagnostico	Verifica con la cámara que cada métrica responda a los movimientos esperados.

3. Especificación del lenguaje (objetivo 1)

3.1 Estructura de un programa

Un programa se compone de uno o más **dominios**. Cada dominio declara sus sensores, variables globales (umbrales), **gestos** (una condición sobre las métricas faciales sostenida durante un tiempo), **secuencias** (gestos en orden dentro de una ventana de tiempo) y **máquinas de estado** que reaccionan a esos eventos. Un fragmento del módulo de conducción:

```
dominio conduccion {
    sensor camara { var fps : int = 30; }
    var umbral_ojo : proporcion = 0.20;

    gesto parpadeo   = ear < umbral_ojo y mar < umbral_bostezo durante 60 ms hasta 400 ms;
    gesto microsueno = ear < umbral_ojo y mar < umbral_bostezo durante 1 s tolerando 100 ms;
    secuencia fatiga = [bostezo, cabeceo] en 10 s;

    maquina monitor inicial normal {
        estado normal {
            cuando microsueno ir_a alerta;
```

```
            mientras perclos > umbral_perclos { ir_a somnoliento; }
        }
        estado alerta {
            al_entrar { alarma("¡Despierte!"); registrar("microsueño"); }
            cuando atento ir_a normal;
        }
    }
}
```

Los programas completos de ambos módulos están en los anexos A y B.

3.2 Componentes léxicos

Tabla 3. Componentes léxicos.

Categoría	Definición
Identificador	[a-zA-Z_][a-zA-Z0-9_]* que no sea palabra reservada
Entero / decimal	[0-9]+ y [0-9]+.[0-9]+
Cadena	texto entre comillas dobles, con escapes \n \t \" \\
Unidades	ms, s y %, que pueden ir pegadas al número (100ms, 2s, 40%)
Operadores	+ - * / < <= > >= == != = y los lógicos y, o, no
Delimitadores	{ } ( ) [ ] ; , :
Comentarios	// hasta el fin de la línea y /* bloque */
Palabras reservadas	dominio sensor var gesto secuencia maquina inicial estado al_entrar cuando ignorar ir_a durante hasta tolerando en si sino mientras repetir veces int decimal bool string duracion proporcion verdadero falso y o no ms s

3.3 Gramática

La gramática libre de contexto, en notación EBNF:

```
programa     = dominio , { dominio } ;
dominio      = "dominio" ID "{" { sensor | decl_var | gesto | secuencia | maquina } "}" ;
sensor       = "sensor" ID "{" { decl_var } "}" ;
decl_var     = "var" ID ":" tipo "=" expr ";" ;
tipo         = "int" | "decimal" | "bool" | "string" | "duracion" | "proporcion" ;
gesto        = "gesto" ID "=" expr "durante" expr [ "hasta" expr ] [ "tolerando" expr ] ";" ;
secuencia    = "secuencia" ID "=" "[" ID { "," ID } "]" [ "en" expr ] ";" ;
maquina      = "maquina" ID "inicial" ID "{" estado { estado } "}" ;
estado       = "estado" ID "{" { miembro_est } "}" ;
miembro_est  = decl_var | "al_entrar" bloque | "cuando" ID ( "ir_a" ID ";" | bloque )
             | "mientras" expr bloque | "ignorar" ID { "," ID } ";" ;
bloque       = "{" { sentencia } "}" ;
sentencia    = decl_var | ID "=" expr ";" | "si" expr bloque [ "sino" ( si | bloque ) ]
             | "repetir" expr "veces" bloque | "ir_a" ID ";" | llamada ";" ;
expr         = conjuncion { "o" conjuncion } ;
conjuncion   = negacion { "y" negacion } ;
negacion     = "no" negacion | comparacion ;
comparacion  = suma [ ( "<" | "<=" | ">" | ">=" | "==" | "!=" ) suma ] ;
suma         = termino { ( "+" | "-" ) termino } ;
termino      = unario { ( "*" | "/" ) unario } ;
unario       = "-" unario | primario ;
primario     = ( ENTERO | DECIMAL ) [ "ms" | "s" | "%" ] | CADENA | "verdadero" | "falso"
             | ID | llamada | "(" expr ")" ;
```

```
llamada      = ID "(" [ expr { "," expr } ] ")" ;
```

La gramática es LL(1) salvo en *sentencia* cuando inicia con un identificador; ahí se usa un token adicional de anticipación (= indica asignación y (indica llamada). La precedencia, de menor a mayor, es: o, y, no, comparaciones (no asociativas), suma y resta, producto y división, menos unario.

3.4 Tipos y reglas de conversión

Además de los tipos primitivos, el lenguaje tiene **duracion** (se normaliza a milisegundos durante la compilación) y **proporcion** (un real entre 0 y 1 que también puede escribirse como porcentaje). Los gestos y las secuencias se declaran con su propia palabra reservada.

Tabla 4. Reglas de conversión de tipos.

Conversión	Permitida	Observación
int → decimal	Sí	Ensanchamiento
int + decimal	Sí	El resultado es decimal
decimal ↔ proporcion	Sí	Si el valor es constante se verifica que esté en [0, 1]
ms ↔ s	Sí	1 s = 1000 ms; se normaliza en compilación
decimal → int	No	Pérdida de precisión
número → duracion	No	Se exige la unidad: 5 s, no 5
duracion comparada con número	No	Se exige la unidad en la constante

Para conversiones explícitas existen las funciones a_porcentaje(0.4) = 40.0, a_proporcion(40) = 40 %, a_ms(2 s) = 2000.0 y a_s(500 ms) = 0.5.

3.5 Ámbitos

Los ámbitos, de exterior a interior, son: **global** (métricas del sensor, de solo lectura, y funciones predefinidas), **dominio** (variables del dominio y de sus sensores, gestos, secuencias y máquinas), **máquina** (nombres de sus estados), **estado** (variables locales, que se reinician cada vez que se entra al estado) y **bloque**. Las condiciones de los gestos se evalúan en el ámbito del dominio, por lo que pueden usar umbrales globales pero no variables locales de un estado.

3.6 Semántica de ejecución

En cada cuadro de video: (1) cada gesto evalúa su condición y avanza su autómata; (2) cada secuencia consume los eventos emitidos por los gestos; (3) cada máquina procesa los eventos en su estado actual (se ejecuta el primer *cuando* que coincide) y después evalúa sus bloques *mientras*. Como pide el anteproyecto, *mientras* se evalúa una sola vez por cuadro y nunca bloquea la ejecución; *repetir N veces* se ejecuta completo dentro del cuadro. Una transición (ir_a) reinicia las variables locales del estado destino y ejecuta su bloque *al_entrar*.

4. Análisis léxico (objetivo 2)

El analizador léxico recorre el texto carácter por carácter. Cada rama del método principal corresponde a un estado de un autómata finito determinista que reconoce una categoría: identificadores y palabras reservadas, números, cadenas, símbolos de uno o dos caracteres (aplicando la regla del lexema más largo, por ejemplo <= antes que <) y comentarios. Cada token guarda su línea y columna para los mensajes de error.

Un caso particular del dominio son las unidades pegadas al número: en 100ms el analizador separa el número de la unidad, mientras que 3abc se reporta como error porque un identificador no puede iniciar con un dígito. También se detectan caracteres no reconocidos, cadenas sin cerrar y comentarios de bloque sin cerrar. Los errores no detienen el análisis: se registran y se continúa con el siguiente token.

```
PS C:\Users\pc\EmoLang> python -m emolang lexico ejemplos/conduccion.emo
   #  LÍNEA:COL  TOKEN          LEXEMA
   0  4:1        DOMINIO        dominio
   1  4:9        ID             conduccion
   2  4:20       LLAVE_IZQ      {
   3  6:5        SENSOR         sensor
   4  6:12       ID             camara
   5  6:19       LLAVE_IZQ      {
   6  7:9        VAR            var
   7  7:13       ID             fps
   8  7:17       DOS_PUNTOS     :
   9  7:19       INT            int
  10  7:23       ASIGNACION     =
  11  7:25       ENTERO         30
  12  7:27       PUNTO_COMA     ;
  13  8:5        LLAVE_DER      }
  14  11:5       VAR            var
  15  11:9       ID             umbral_ojo
  16  11:24      DOS_PUNTOS     :
  17  11:26      PROPORCION     proporcion
  18  11:37      ASIGNACION     =
  19  11:39      DECIMAL        0.20
  20  11:43      PUNTO_COMA     ;
  21  13:5       VAR            var
  22  13:9       ID             umbral_bostezo
  23  13:24      DOS_PUNTOS     :
  24  13:26      PROPORCION     proporcion
  25  13:37      ASIGNACION     =
  26  13:39      ENTERO         33
  27  13:41      PORCIENTO      %
  28  13:42      PUNTO_COMA     ;
```

Figura 2. Primeros tokens del programa de conducción.

5. Análisis sintáctico (objetivo 2)

El analizador sintáctico es descendente recursivo: cada producción de la gramática es un método. Produce un árbol de sintaxis abstracta (AST) cuyos nodos están definidos en nodos.py; las constantes con unidad ya llegan normalizadas al árbol (2 s se guarda como 2000 ms y 40 % como 0.4).

Ante un error se aplica recuperación en modo pánico: se reporta el error y se descartan tokens hasta un punto de sincronización (punto y coma, llave de cierre o el inicio de la siguiente declaración). Así se informan varios errores en una sola compilación en lugar de detenerse en el primero. El analizador también da mensajes propios del lenguaje, por ejemplo cuando a un gesto le falta la cláusula durante o cuando se usa mientras fuera de un estado.

```
PS C:\Users\pc\EmoLang> python -m emolang sintactico ejemplos/rehabilitacion.emo
Programa()
└── dominios: [1]
    └── Dominio(nombre='rehabilitacion')
        ├── sensores: [1]
        │   └── Sensor(nombre='camara')
        │       └── variables: [1]
        │           └── DeclVar(nombre='fps', tipo_decl='int')
        │               └── valor: Literal(valor=30, tipo_lit='int')
        ├── variables: [5]
        │   ├── DeclVar(nombre='meta', tipo_decl='int')
        │   │   └── valor: Literal(valor=8, tipo_lit='int')
        │   ├── DeclVar(nombre='umbral_sonrisa', tipo_decl='proporcion')
        │   │   └── valor: Literal(valor=0.4, tipo_lit='proporcion')
        │   ├── DeclVar(nombre='umbral_reposo', tipo_decl='proporcion')
        │   │   └── valor: Literal(valor=0.15, tipo_lit='decimal')
        │   ├── DeclVar(nombre='simetria_minima', tipo_decl='proporcion')
        │   │   └── valor: Literal(valor=0.8, tipo_lit='proporcion')
        │   └── DeclVar(nombre='sostener', tipo_decl='duracion')
        │       └── valor: Literal(valor=500.0, tipo_lit='duracion')
        ├── gestos: [2]
        │   ├── Gesto(nombre='sonreir')
        │   │   ├── condicion: Binaria(op='>')
        │   │   │   ├── izq: Identificador(nombre='sonrisa')
        │   │   │   └── der: Identificador(nombre='umbral_sonrisa')
        │   │   ├── minimo: Identificador(nombre='sostener')
        │   │   └── tolerancia: Literal(valor=100.0, tipo_lit='duracion')
        │   └── Gesto(nombre='reposo')
        │       ├── condicion: Binaria(op='<')
        │       │   ├── izq: Identificador(nombre='sonrisa')
        │       │   └── der: Identificador(nombre='umbral_reposo')
        │       └── minimo: Literal(valor=300.0, tipo_lit='duracion')
        └── maquinas: [1]
            └── Maquina(nombre='ejercicio', inicial='preparacion')
                └── estados: [4]
                    ├── Estado(nombre='preparacion')
                    │   ├── al_entrar: Bloque()
                    │   │   └── sentencias: [1]
                    │   │       └── SentenciaLlamada()
                    │   │           └── llamada: Llamada(nombre='mostrar')
                    │   │               └── args: [1]
                    │   │                   └── Literal(valor='Relaje el rostro para comenzar', tipo_lit='string')
                    │   ├── cuandos: [1]
                    │   │   └── Cuando(evento='reposo', destino='esperando')
                    │   └── ignorados: [1]
                    │       └── Identificador(nombre='sonreir')
                    ├── Estado(nombre='esperando')
```

Figura 3. Árbol de sintaxis abstracta del programa de rehabilitación (inicio).

6. Análisis semántico (objetivo 3)

El analizador semántico recorre el AST con una tabla de símbolos organizada como una cadena de ámbitos. Para cada símbolo guarda su nombre, categoría (variable, gesto, secuencia, máquina, estado), tipo, ámbito, línea y, cuando se puede calcular en compilación, su valor inicial. Ese valor se obtiene por plegado de constantes y lo usan después las validaciones de dominio.

Las verificaciones que realiza son: identificadores no declarados o redeclarados en el mismo ámbito; uso de nombres reservados (métricas y funciones); asignación a métricas, gestos o estados; compatibilidad de tipos en asignaciones, operadores y argumentos de funciones; que las condiciones de si, mientras y de los gestos sean bool; que durante, hasta, tolerando y la ventana de una secuencia sean de tipo duracion; que ir_a apunte a un estado de la misma máquina; y que cuando e ignorar se refieran a gestos o secuencias. Además anota en cada expresión del AST su tipo, que el motor usa para mostrar valores (por ejemplo, una proporción se imprime como 16.5 %).

```
PS C:\Users\pc\EmoLang> python -m emolang semantico ejemplos/conduccion.emo
TABLA DE SÍMBOLOS
NOMBRE             CATEGORÍA  TIPO        ÁMBITO                      LÍNEA  VALOR INICIAL
fps                variable   int         sensor camara                   5  30
umbral_ojo         variable   proporcion  dominio conduccion              9  0.2
umbral_bostezo     variable   proporcion  dominio conduccion             11  0.33
umbral_perclos     variable   proporcion  dominio conduccion             12  0.15
cabeceo_grados     variable   decimal     dominio conduccion             13  10.0
giro_grados        variable   decimal     dominio conduccion             14  20.0
alertas            variable   int         dominio conduccion             15  0
parpadeo           gesto      gesto       dominio conduccion             20
microsueno         gesto      gesto       dominio conduccion             21
bostezo            gesto      gesto       dominio conduccion             22
cabeceo            gesto      gesto       dominio conduccion             23
distraccion        gesto      gesto       dominio conduccion             24
atento             gesto      gesto       dominio conduccion             25
fatiga             secuencia  secuencia   dominio conduccion             28
monitor            maquina    maquina     dominio conduccion             31
normal             estado     estado      maquina monitor                33
somnoliento        estado     estado      maquina monitor                50
distraido          estado     estado      maquina monitor                63
alerta             estado     estado      maquina monitor                70
parpadeos          variable   int         estado normal                  34  0


0 error(es), 0 advertencia(s)
PS C:\Users\pc\EmoLang>
```

Figura 4. Tabla de símbolos del programa de conducción.

7. Validaciones propias del dominio (objetivo 4)

Esta fase revisa reglas que un lenguaje de propósito general no podría verificar, porque dependen del significado de las métricas faciales y del tiempo. Se ejecuta aunque haya errores semánticos, para reportar todos los problemas de una vez.

Tabla 5. Validaciones de dominio.

Validación	Técnica	Ejemplo de mensaje
Umbral fuera de rango	Cada métrica tiene un rango válido y uno típico (ear en [0, 1], típico 0 a 0.45). Fuera del válido es error; fuera del típico, advertencia.	umbral fuera de rango: 'ear' toma valores en [0, 1] y se comparó con 1.2
Gesto imposible	Análisis de intervalos sobre la condición (ver abajo); además duración mínima mayor que la máxima, o máxima menor que un cuadro.	gesto imposible: la condición de 'imposible1' nunca puede cumplirse
Secuencia imposible	Contiene un gesto imposible, o la ventana no alcanza para repetir un mismo gesto.	secuencia imposible: 'rep' necesita al menos 2200 ms
Estado inalcanzable	Recorrido en anchura (BFS) del grafo de transiciones desde el estado inicial.	estado inalcanzable: 'huerfano' no puede alcanzarse...
Transiciones contradictorias	Un mismo evento con transiciones incondicionales a destinos distintos en el mismo estado (no determinismo).	el evento 'bien' lleva a b, c
Eventos ignorados (nota)	Un evento que cambia de estado en unos estados y se ignora en otros sin declararlo con ignorar.	'distraccion' cambia de estado en normal pero se ignora en somnoliento

Análisis de intervalos. La condición de un gesto se transforma a forma normal disyuntiva sobre intervalos: cada comparación métrica-constante se convierte en un intervalo (ear < 0.1 da [0, 0.1)), la conjunción intersecta intervalos de la misma métrica, la disyunción los une y la negación se propaga con las leyes de De Morgan. Si todas las alternativas resultantes quedan vacías al intersectarlas con el rango válido de cada métrica, la condición es insatisfacible. Por ejemplo, ear < 0.1 y ear > 0.3 produce un intervalo vacío.

Otras advertencias: gestos o secuencias declarados y no usados, condiciones siempre verdaderas, duraciones menores a un cuadro, transiciones que dependen de un gesto imposible y una tolerancia mayor o igual a la duración del gesto. La Figura 5 muestra el resultado con un programa escrito con errores intencionales.

```
PS C:\Users\pc\EmoLang> python -m emolang semantico ejemplos/errores.emo
huerfano           estado     estado      maquina m                      28

  [semantico] error en 6:5: umbral fuera de rango: una proporcion debe estar entre 0 y 1 (0% y 100%), se obtuvo
1.5
  [semantico] error en 7:5: no se puede asignar int a la variable 'espera' de tipo duracion
  [semantico] error en 8:5: no se puede asignar decimal a la variable 'contador' de tipo int
  [dominio] error en 11:5: gesto imposible: la condición de 'imposible1' nunca puede cumplirse
  [dominio] error en 12:5: gesto imposible: 'imposible2' dura al menos 2000 ms pero como máximo 1000 ms
  [dominio] advertencia en 12:5: 'imposible2' se declara pero ningún 'cuando' ni secuencia lo usa
  [dominio] error en 13:5: gesto imposible: la condición de 'fuera' nunca puede cumplirse
  [dominio] advertencia en 13:5: 'fuera' se declara pero ningún 'cuando' ni secuencia lo usa
  [dominio] error en 13:28: umbral fuera de rango: 'ear' toma valores en [0, 1] y se comparó con 1.2
  [dominio] advertencia en 14:5: 'raro' se declara pero ningún 'cuando' ni secuencia lo usa
  [semantico] error en 14:44: 'durante' requiere una duración con unidad (ms o s), se recibió int
  [dominio] nota en 17:5: en la máquina 'm', el evento 'bien' cambia de estado en a, c, huerfano pero se ignora
en b (si es intencional escriba 'ignorar bien;' en esos estados)
  [dominio] nota en 17:5: en la máquina 'm', el evento 'imposible1' cambia de estado en b pero se ignora en a, c
, huerfano (si es intencional escriba 'ignorar imposible1;' en esos estados)
  [dominio] error en 20:13: transiciones contradictorias en 'a': el evento 'bien' lleva a b, c
  [semantico] error en 21:13: evento 'noexiste' no declarado
  [dominio] advertencia en 24:13: la transición 'cuando imposible1' del estado 'b' nunca se dispara: el evento e
s imposible
  [semantico] error en 25:25: no se puede asignar string a 'ok' de tipo decimal
  [dominio] error en 28:9: estado inalcanzable: 'huerfano' no puede alcanzarse desde el estado inicial 'a'

12 error(es), 4 advertencia(s), 2 nota(s)
PS C:\Users\pc\EmoLang>
```

Figura 5. Diagnósticos del programa ejemplos/errores.emo (errores intencionales).

8. Generación de autómatas (objetivo 5)

8.1 Autómata de un gesto

Un gesto se traduce a un autómata finito determinista sobre el alfabeto Σ = {1, 0}, donde 1 significa que la condición se cumple en el cuadro actual. La duración se convierte a cuadros con N = □D · fps / 1000□. El estado q_i significa que la condición lleva i cuadros cumplidos. Con 1 se avanza de q_i a q_{i+1}; con 0 se regresa a q_0. El autómata es una máquina de Mealy: la transición que llega a q_N emite el evento del gesto (marcado como / g en las figuras). Así se modela lo que plantea el anteproyecto: un ojo cerrado durante un cuadro es un parpadeo; durante un segundo, un posible microsueño.

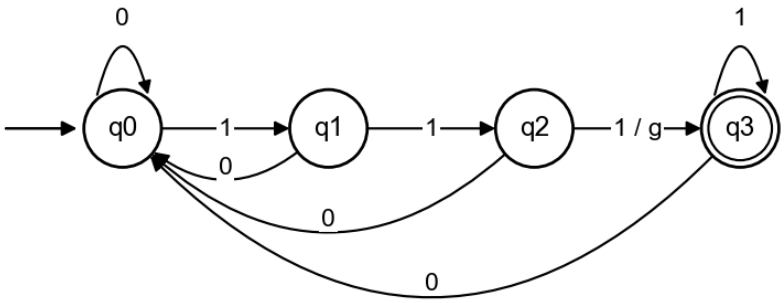


Figura 6. AFD de un gesto de 3 cuadros (100 ms a 30 fps). Doble círculo: estado de aceptación.

Si el gesto tiene duración máxima (hasta M), se agregan los estados hasta q_M y un estado de exceso q_X, y el evento se emite al soltar la condición sólo si duró entre N y M cuadros. Así se distingue un parpadeo (60 a 400 ms) de un microsueño.

8.2 Tolerancia a interrupciones

Durante las pruebas se observó que un solo cuadro con ruido reinicia el conteo de un gesto sostenido (sección 11.3). Por eso se agregó la cláusula tolerando T, que perdona hasta K = round(T · fps / 1000) cuadros seguidos sin cumplirse la condición. Se agregan los estados q_i~g, que significan i cuadros cumplidos y g cuadros de interrupción: con 1 se continúa el conteo y con 0 la interrupción crece hasta superar K, momento en que se regresa a q_0. El resultado sigue siendo un autómata finito.

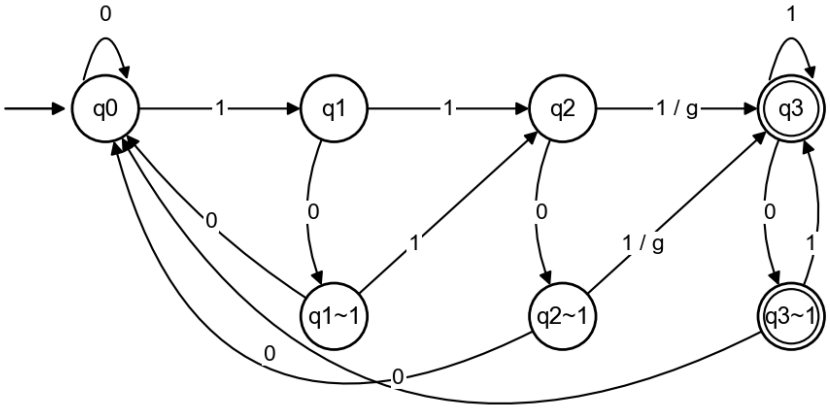


Figura 7. El mismo gesto con tolerancia de un cuadro (estados q_i~1).

El costo es el número de estados, que crece a 1 + N(K + 1) en un gesto sin máximo. La Tabla 6 muestra los tamaños en el programa de conducción.

Tabla 6. Número de estados de los autómatas de gesto generados (30 fps).

Gesto	Definición	Estados
parpadeo	durante 60 ms hasta 400 ms	14
cabeceo	durante 300 ms tolerando 100 ms	37
microsueno	durante 1 s tolerando 100 ms	121
bostezo	durante 1.5 s tolerando 200 ms	316
distraccion	durante 2 s tolerando 200 ms	421
atento	durante 3 s tolerando 200 ms	631

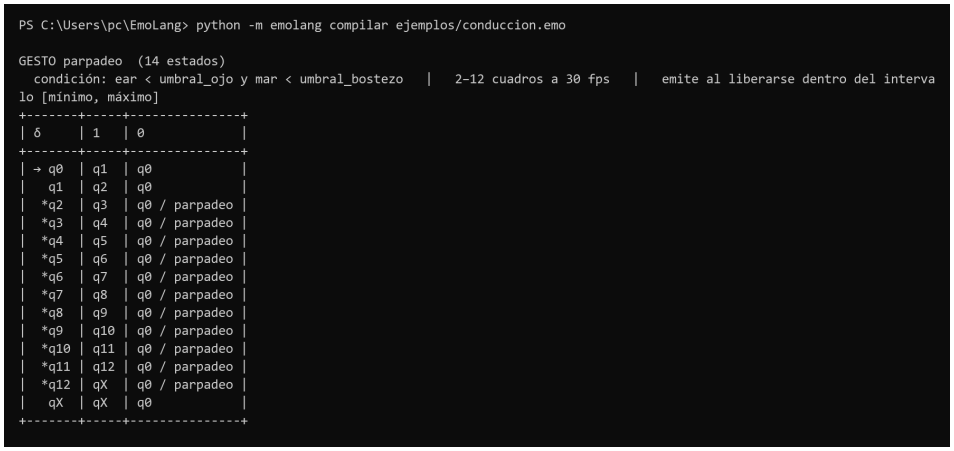


Figura 8. Tabla de transiciones generada para el gesto parpadeo. → marca el estado inicial, * los de aceptación y / el evento emitido.

8.3 Secuencias

Una secuencia como [bostezo, cabeceo] en 10 s se traduce a un autómata de reconocimiento de patrones construido con el algoritmo de Knuth, Morris y Pratt, cuyo alfabeto son los eventos de los gestos que la componen. Si entre el primer evento y el último pasa más tiempo que la ventana, el autómata regresa a s_0.

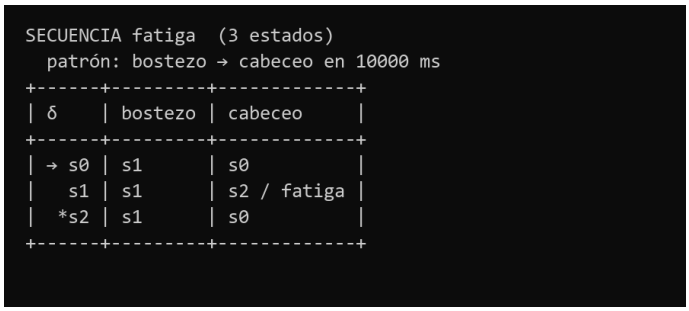


Figura 9. Autómata de la secuencia fatiga.

8.4 Máquinas de estado

Las máquinas declaradas se traducen directamente: sus estados son los declarados y su alfabeto, los eventos que aparecen en sus cuando. Las transiciones dentro de un si llevan una guarda entre corchetes, y las que ocurren desde al_entrar o mientras se marcan con λ (transición espontánea). Un evento que ejecuta acciones sin cambiar de estado aparece como un lazo con salida «acciones».

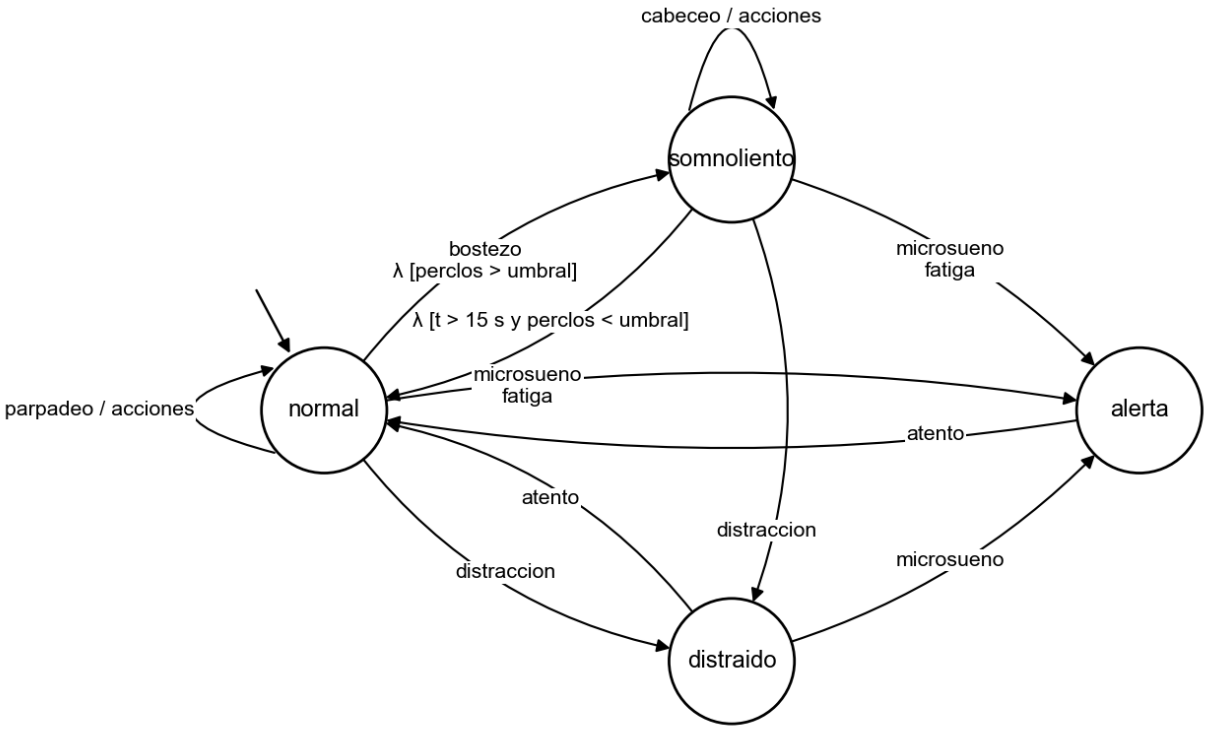


Figura 10. Diagrama generado de la máquina monitor del módulo de conducción.

Para cada autómata el comando compilar produce la tabla de transiciones en texto, un archivo Markdown con tablas y diagramas Mermaid, un archivo .dot para Graphviz y un JSON con la definición completa, que es lo que carga el motor.

9. Motor de ejecución y sensor de cámara (objetivo 6)

9.1 Ciclo por cuadro

El motor no vuelve a interpretar las condiciones temporales: evalúa la condición de cada gesto para obtener el símbolo 1 o 0 y consulta la tabla de transiciones generada. Las secuencias y las máquinas también se ejecutan desde sus tablas. Las sentencias de los bloques (asignaciones, si, repetir, llamadas) sí se interpretan sobre el AST. El tiempo de procesamiento medido es de 0.05 a 0.1 ms por cuadro, muy por debajo de los 33 ms disponibles a 30 fps.

9.2 Velocidad de la cámara

La cámara usada en las pruebas entrega unos 10 cuadros por segundo, no 30. Como los autómatas cuentan cuadros, un gesto de 1 s tardaba en realidad unos 3 s en detectarse. Se resolvió con una normalización: el motor recibe cuadros a ritmo fijo (el fps declarado en el sensor del programa) y, si la cámara se atrasa, repite la medición más reciente. Así, 1 s en el lenguaje es 1 s real sin importar la cámara. Una prueba unitaria reproduce este caso.

9.3 Métricas faciales

De los puntos que entrega MediaPipe se calculan las métricas que usan los programas:

- **ear** (razón de aspecto del ojo, Soukupová y Čech): (|p2 − p6| + |p3 − p5|) / (2 |p1 − p4|) con seis puntos alrededor de cada ojo; se promedian ambos ojos.
- **mar** (razón de aspecto de la boca): distancia vertical promedio entre los labios internos dividida entre la distancia entre las comisuras.
- **sonrisa**, **sonrisa_izq**, **sonrisa_der** y **cejas**: coeficientes de expresión (blendshapes) de MediaPipe.
- **pitch**, **yaw**, **roll**: ángulos de la cabeza obtenidos de la matriz de transformación facial, relativos a una postura neutra medida al inicio.
- **simetria**: 1 menos la diferencia relativa entre el lado izquierdo y el derecho.
- **perclos**: proporción de cuadros con ojos cerrados en los últimos 60 s (Wierwille et al.).

La postura neutra se calibra durante los primeros 2 s: el sistema espera a detectar el rostro y sólo acepta la calibración si la cabeza permanece quieta; durante la ejecución se puede recalibrar con la tecla r. Además del sensor de cámara existen un sensor simulado (genera métricas a partir de un guion de eventos y conoce la verdad de terreno) y un sensor que reproduce grabaciones en CSV.

10. Módulos de dominio (objetivo 7)

Los módulos implementan las funciones con efectos y el resumen final. Se eligen según el nombre del dominio.

- **Conducción**: alarma(mensaje) activa un aviso sonoro y visual sobre el video, detener_alarma() lo apaga y registrar(mensaje) agrega un evento a la bitácora, que se exporta a CSV. El resumen informa alarmas emitidas, tiempo en alarma y gestos detectados.
- **Rehabilitación**: contar_repeticion() registra una repetición con la simetría del momento y repeticiones() devuelve el conteo. El resumen informa repeticiones, simetría promedio, mínima y máxima, y el ritmo promedio entre repeticiones.

```
PS C:\Users\pc\EmoLang> python -m emolang ejecutar ejemplos/conduccion.emo
  [   0.00 s] estado     monitor: inicio → normal
  [   2.97 s] gesto      atento
  [   6.33 s] registro   PERCLOS alto: 15.2%
  [   6.33 s] estado     monitor: normal → somnoliento
  [   6.33 s] mensaje    Conductor somnoliento, vigilando...
  [   6.93 s] gesto      bostezo
  [  11.97 s] gesto      microsueno
  [  11.97 s] estado     monitor: somnoliento → alerta
  [  11.97 s] ALARMA     ¡Despierte! Riesgo de microsueño
  [  11.97 s] registro   alerta #1
  [  18.93 s] gesto      bostezo
  [  21.97 s] gesto      cabeceo
  [  21.97 s] secuencia  fatiga
  [  25.17 s] gesto      atento
  [  25.17 s] estado     monitor: alerta → normal
  [  25.20 s] registro   PERCLOS alto: 21.8%
  [  25.20 s] estado     monitor: normal → somnoliento
  [  25.20 s] mensaje    Conductor somnoliento, vigilando...
  [  29.97 s] gesto      microsueno
  [  29.97 s] estado     monitor: somnoliento → alerta
  [  29.97 s] ALARMA     ¡Despierte! Riesgo de microsueño
  [  29.97 s] registro   alerta #2
  [  36.17 s] gesto      distraccion
  [  39.30 s] gesto      atento
  [  39.30 s] estado     monitor: alerta → normal
  [  39.33 s] registro   PERCLOS alto: 21.2%
  [  39.33 s] estado     monitor: normal → somnoliento
  [  39.33 s] mensaje    Conductor somnoliento, vigilando...

RESUMEN DEL MÓDULO DE DOMINIO
  alarmas_emitidas: 2
  tiempo_en_alarma_s: 22.53
  gestos_detectados: {'parpadeo': 9, 'atento': 3, 'bostezo': 2, 'microsueno': 2, 'cabeceo': 1, 'distraccion': 1}
  eventos_registrados: 5
  cuadros procesados: 1200  |  latencia de procesamiento: media 0.048 ms, p95 0.066 ms por cuadro
PS C:\Users\pc\EmoLang>
```

Figura 11. Ejecución del programa de conducción con el escenario simulado.

```
PS C:\Users\pc\EmoLang> python -m emolang ejecutar ejemplos/rehabilitacion.emo
  [  19.83 s] gesto      reposo
  [  19.83 s] estado     ejercicio: contraccion → esperando
  [  22.67 s] gesto      sonreir
  [  22.67 s] estado     ejercicio: esperando → contraccion
  [  22.67 s] mensaje    Repetición 6 de 8 - simetría 71.8%
  [  22.67 s] registro   simetría baja en repetición 6
  [  23.80 s] gesto      reposo
  [  23.80 s] estado     ejercicio: contraccion → esperando
  [  26.63 s] gesto      sonreir
  [  26.63 s] estado     ejercicio: esperando → contraccion
  [  26.63 s] mensaje    Repetición 7 de 8 - simetría 96.9%
  [  27.83 s] gesto      reposo
  [  27.83 s] estado     ejercicio: contraccion → esperando
  [  30.63 s] gesto      sonreir
  [  30.63 s] estado     ejercicio: esperando → contraccion
  [  30.63 s] mensaje    Repetición 8 de 8 - simetría 91.1%
  [  31.83 s] gesto      reposo
  [  31.83 s] estado     ejercicio: contraccion → terminado
  [  31.83 s] mensaje    ¡Ejercicio completo!
  [  31.83 s] registro   fin de sesión
  [  31.83 s] registro   fin de sesión
  [  31.83 s] registro   fin de sesión
  [  34.67 s] gesto      sonreir
  [  35.83 s] gesto      reposo
  [  38.63 s] gesto      sonreir
  [  39.83 s] gesto      reposo

RESUMEN DEL MÓDULO DE DOMINIO
  repeticiones: 8
  simetria_promedio_%: 87.9
  simetria_minima_%: 71.8
  simetria_maxima_%: 96.9
  ritmo_promedio_s: 4.0
  cuadros procesados: 1350  |  latencia de procesamiento: media 0.062 ms, p95 0.039 ms por cuadro
PS C:\Users\pc\EmoLang>
```

Figura 12. Final de la ejecución del programa de rehabilitación: conteo de repeticiones y simetría.

11. Pruebas y resultados (objetivo 8)

11.1 Pruebas unitarias

Hay 35 pruebas automáticas que cubren cada fase. Todas pasan en Python 3.14 y en Python 3.11.

Tabla 7. Pruebas unitarias por grupo.

Grupo	Pruebas	Qué verifican
Léxico	5	Tokens, unidades pegadas, operadores dobles, comentarios, errores y posiciones.
Sintáctico	5	Ejemplos válidos, precedencia, literales con unidad, recuperación de errores.
Semántico	6	Conversiones válidas e inválidas, ámbitos, redeclaraciones, funciones, tabla.
Validaciones	6	Umbrales, gestos y secuencias imposibles, estados inalcanzables, contradicciones, eventos ignorados, advertencias.
Autómatas	7	Ejecución de los AFD de gestos (con máximo y tolerancia), KMP y formatos.
Motor	6	Transiciones, mientras no bloqueante, cámara lenta, escenarios completos.

```
PS C:\Users\pc\EmoLang> python -m unittest discover -s tests -t . -v
test_generacion_y_formatos (tests.test_emolang.TestAutomatas.test_generacion_y_formatos) ... ok
test_gesto_con_maximo_parpadeo (tests.test_emolang.TestAutomatas.test_gesto_con_maximo_parpadeo) ... ok
test_gesto_con_maximo_y_tolerancia (tests.test_emolang.TestAutomatas.test_gesto_con_maximo_y_tolerancia) ... ok
test_gesto_con_tolerancia (tests.test_emolang.TestAutomatas.test_gesto_con_tolerancia) ... ok
test_gesto_minimo (tests.test_emolang.TestAutomatas.test_gesto_minimo) ... ok
test_secuencia_kmp (tests.test_emolang.TestAutomatas.test_secuencia_kmp) ... ok
test_tolerancia_en_el_lenguaje (tests.test_emolang.TestAutomatas.test_tolerancia_en_el_lenguaje) ... ok
test_errores_lexicos (tests.test_emolang.TestLexico.test_errores_lexicos) ... ok
test_operadores_dobles_y_comentarios (tests.test_emolang.TestLexico.test_operadores_dobles_y_comentarios) ... ok
test_palabras_reservadas_e_identificadores (tests.test_emolang.TestLexico.test_palabras_reservadas_e_identificadores) ... ok
test_posiciones (tests.test_emolang.TestLexico.test_posiciones) ... ok
test_unidades_pegadas_y_porcentaje (tests.test_emolang.TestLexico.test_unidades_pegadas_y_porcentaje) ... ok
test_camara_lenta_se_normaliza_al_fps_del_programa (tests.test_emolang.TestMotor.test_camara_lenta_se_normaliza_al_fps_del_p
rograma) ... ok
test_conduccion_simulada (tests.test_emolang.TestMotor.test_conduccion_simulada) ... ok
test_escenario_normal_sin_alarmas (tests.test_emolang.TestMotor.test_escenario_normal_sin_alarmas) ... ok
test_mientras_no_bloquea (tests.test_emolang.TestMotor.test_mientras_no_bloquea) ... ok
test_rehabilitacion_cuenta_repeticiones (tests.test_emolang.TestMotor.test_rehabilitacion_cuenta_repeticiones) ... ok
test_transicion_por_gesto_y_variables (tests.test_emolang.TestMotor.test_transicion_por_gesto_y_variables) ... ok
test_ambitos (tests.test_emolang.TestSemantico.test_ambitos) ... ok
test_conversiones_validas (tests.test_emolang.TestSemantico.test_conversiones_validas) ... ok
test_errores_de_tipo (tests.test_emolang.TestSemantico.test_errores_de_tipo) ... ok
test_metricas_solo_lectura_y_funciones (tests.test_emolang.TestSemantico.test_metricas_solo_lectura_y_funciones) ... ok
test_redeclaracion_y_reservados (tests.test_emolang.TestSemantico.test_redeclaracion_y_reservados) ... ok
test_tabla_de_simbolos (tests.test_emolang.TestSemantico.test_tabla_de_simbolos) ... ok
test_ejemplos_validos (tests.test_emolang.TestSintactico.test_ejemplos_validos) ... ok
test_literales_de_dominio (tests.test_emolang.TestSintactico.test_literales_de_dominio) ... ok
test_mientras_fuera_de_estado (tests.test_emolang.TestSintactico.test_mientras_fuera_de_estado) ... ok
test_precedencia (tests.test_emolang.TestSintactico.test_precedencia) ... ok
test_recuperacion_reporta_varios_errores (tests.test_emolang.TestSintactico.test_recuperacion_reporta_varios_errores) ... ok
test_advertencias (tests.test_emolang.TestValidacionesDominio.test_advertencias) ... ok
test_estados_inalcanzables_y_contradictorios (tests.test_emolang.TestValidacionesDominio.test_estados_inalcanzables_y_contra
dictorios) ... ok
test_eventos_ignorados_e_ignorar (tests.test_emolang.TestValidacionesDominio.test_eventos_ignorados_e_ignorar) ... ok
test_gestos_imposibles (tests.test_emolang.TestValidacionesDominio.test_gestos_imposibles) ... ok
test_secuencia_imposible_por_ventana (tests.test_emolang.TestValidacionesDominio.test_secuencia_imposible_por_ventana) ... o
k
test_umbral_fuera_de_rango (tests.test_emolang.TestValidacionesDominio.test_umbral_fuera_de_rango) ... ok

----------------------------------------------------------------------
Ran 35 tests in 0.295s

OK
PS C:\Users\pc\EmoLang>
```

Figura 13. Resultado de las pruebas unitarias.

11.2 Evaluación con escenarios simulados

El sensor simulado genera métricas con ruido y conoce cuándo ocurre cada gesto. Se usaron dos escenarios de conducción: uno con bostezos, microsueños, cabeceos y distracciones (40 s), y uno normal con sólo parpadeos (60 s) para medir falsos positivos; además un escenario de rehabilitación con 10 sonrisas. Cada escenario se corrió 20 veces con distinta semilla aleatoria. Una detección es verdadera positiva si ocurre entre el inicio del gesto real y 1.5 s después de su fin. Las magnitudes del simulador se ajustaron con las mediciones de la cámara real (sección 11.4).

Tabla 8. Resultados con datos simulados (20 corridas por escenario).

Gesto	VP	FP	FN	Precisión	Recall	Latencia media
parpadeo	447	0	0	100 %	100 %	191 ms
microsueno	40	0	0	100 %	100 %	967 ms
bostezo	40	0	0	100 %	100 %	1,923 ms
cabeceo	20	0	0	100 %	100 %	473 ms
distraccion	20	0	0	100 %	100 %	2,172 ms
sonreir (rehabilitación)	200	0	0	100 %	100 %	646 ms

La latencia de detección incluye la duración mínima que exige cada gesto (un microsueño necesita 1 s), por lo que no es un retraso del sistema. Estos resultados deben leerse como una cota superior: el simulador es más regular que una persona real. Su utilidad principal es comparar versiones de un programa en las mismas condiciones, como en la siguiente sección.

11.3 Robustez ante ruido

Para medir el efecto de los cambios hechos al programa de conducción durante el desarrollo, se comparó la versión inicial (sin tolerando y sin las condiciones sobre mar) con la versión actual, aumentando el ruido del simulador al doble y al triple. Ambas se midieron con el mismo simulador.

Tabla 9. Versión inicial contra versión actual del programa de conducción (exactitud = VP / (VP + FP + FN)).

Ruido	Versión	Exactitud total	Recall cabeceo	FP parpadeo	FP microsueño
×1	Inicial	87.6 %	95 %	39	40
	Actual	100 %	100 %	0	0
×2	Inicial	78.6 %	80 %	126	23
	Actual	99.3 %	100 %	4	0
×3	Inicial	68.2 %	25 %	239	3
	Actual	97.6 %	100 %	13	0

La mejora se obtuvo modificando sólo el programa EmoLang, sin cambiar el motor: tolerando evita que el ruido reinicie los gestos sostenidos, y agregar y mar < umbral_bostezo a parpadeo y microsueño evita confundirlos con los ojos entrecerrados durante un bostezo. Es el tipo de ajuste que el lenguaje busca dejar en manos de quien conoce el dominio.

11.4 Pruebas con cámara real

Se agregó el comando diagnostico, que pide a la persona una serie de movimientos (mirar al frente, bajar y subir la cabeza, girar a cada lado, cerrar los ojos, abrir la boca y sonreír), promedia las métricas de cada paso y verifica que respondan en la dirección esperada. La primera ejecución (Figura 14) reveló tres problemas.

```
PS C:\Users\pc\EmoLang> .venv\Scripts\python -m emolang diagnostico
Diagnóstico de la cámara: 8 pasos de 4 s (más 2 s de calibración).
Siga las instrucciones de la ventana; un pitido marca cada paso nuevo.

PROMEDIOS POR PASO
paso              ear       mar   sonrisa     pitch       yaw      roll  simetria
frente           0.28      0.00      0.00     -2.00     -0.61      0.95      0.97
abajo            0.18      0.01      0.00      2.65     -1.44      1.64      0.97
arriba           0.34      0.01      0.00     -5.35     -4.72      0.72      0.99
izquierda        0.31      0.01      0.01     -7.48     14.57      0.23      0.89
derecha          0.28      0.02      0.00     -3.34    -13.62     -4.40      0.93
ojos             0.15      0.01      0.00     -5.51     -2.40      0.96      0.98
boca             0.15      0.57      0.03     -7.91     -3.59      0.86      0.95
sonrisa          0.18      0.30      0.93     -2.71     -3.78      0.67      0.95

VERIFICACIÓN
  ✗ Baje la cabeza (mire al piso): pitch debe AUMENTAR al bajar la cabeza
  ✗ Suba la cabeza (mire al techo): pitch debe DISMINUIR al subir la cabeza
  ✓ Gire la cabeza a SU izquierda: yaw debe cambiar más de 15 grados
  ✗ Gire la cabeza a SU derecha: yaw debe cambiar más de 15 grados
  ✓ Cierre los ojos (siga escuchando el pitido): ear debe bajar de 0.20 con los ojos cerrados
  ✓ Abra la boca grande (como bostezo): mar debe superar 0.50
  ✓ Sonria ampliamente: sonrisa debe superar 0.40
  ✓ yaw tiene signo opuesto a la izquierda (+15) y a la derecha (-13)

Cuadros sin rostro: 8/321 (2%)
3 verificación(es) fallaron: copie este reporte para revisarlo.
```

Figura 14. Primer diagnóstico con la cámara.

Tabla 10. Hallazgos de las pruebas con cámara.

Hallazgo	**Evidencia**	**Cambio realizado**
La cámara entrega ~10 fps, no 30	321 cuadros en ~34 s	Normalización del ritmo de cuadros en el motor (9.2)
Al bostezar o sonreír se entrecierran los ojos	ear = 0.15 con la boca abierta y 0.18 al sonreír (umbral 0.20)	Condición y mar < umbral_bostezo en parpadeo y microsueño; simulador ajustado
La postura neutra quedó mal calibrada	pitch = 10.7° mirando al frente en el segundo diagnóstico	Calibración que espera a que la cabeza esté quieta y recalibración con r
La boca abierta da mar ≈ 0.38	El umbral de bostezo era 0.55	Umbral de bostezo a 33 %
El giro de cabeza da 23° a 33°	Umbral de distracción de 30°	Umbral a 20° (variable giro_grados)

```
PS C:\Users\pc\EmoLang> .venv\Scripts\python -m emolang diagnostico
Diagnóstico de la cámara: 8 pasos de 4 s (más 2 s de calibración).
Siga las instrucciones de la ventana; un pitido marca cada paso nuevo.

PROMEDIOS POR PASO
paso              ear       mar   sonrisa     pitch       yaw      roll  simetria
frente           0.28      0.01      0.00     10.74     -2.94      0.50      1.00
abajo            0.21      0.01      0.00     18.68     -2.24      0.60      0.99
arriba           0.29      0.02      0.00     -1.48     -3.48     -0.19      0.99
izquierda        0.30      0.03      0.03     -2.16     19.59     -4.11      0.91
derecha          0.37      0.02      0.01     -9.46    -35.53      3.98      0.90
ojos             0.14      0.01      0.00     -1.78     -4.15      3.43      0.98
boca             0.19      0.38      0.01      0.07     -3.35      1.39      0.99
sonrisa          0.22      0.25      0.79      9.45     -1.68      1.15      0.95

VERIFICACIÓN
  ✗ Baje la CABEZA, barbilla al pecho (no solo los ojos): pitch debe AUMENTAR más de 10 grados (cambió +7.9°)
  ✓ Suba la CABEZA, mire al techo: pitch debe DISMINUIR más de 10 grados (cambió -12.2°)
  ✓ Gire la cabeza a SU izquierda (como ver el espejo): yaw debe cambiar más de 15 grados (cambió +22.5°)
  ✓ Gire la cabeza a SU derecha (como ver el espejo): yaw debe cambiar más de 15 grados (cambió -32.6°)
  ✓ Cierre los ojos (siga escuchando el pitido): ear debe bajar de 0.20 con los ojos cerrados
  ✗ Abra la boca grande (como bostezo): mar debe superar 0.50
  ✓ Sonria ampliamente: sonrisa debe superar 0.40
  ✓ yaw tiene signo opuesto a la izquierda (+23) y a la derecha (-33)

Velocidad real de la cámara: 10.0 fps (el motor la normaliza al fps del programa)

Cuadros sin rostro: 1/320 (0%)
2 verificación(es) fallaron: copie este reporte para revisarlo.
```

Figura 15. Segundo diagnóstico, con movimientos amplios.

Después se ejecutó el programa de conducción completo con la cámara (Figura 16). Se detectaron bostezo, microsueño, cabeceo, la secuencia fatiga y distracción. La distracción se detectó tres veces pero sólo la primera generó alarma, porque en ese momento el monitor estaba en el estado normal; en somnoliento el programa no tenía regla para ese evento. No era una falla del detector sino de las reglas del programa. Se corrigió el ejemplo y se agregó al compilador la nota de eventos ignorados y la instrucción ignorar (sección 12), para que este tipo de omisión se detecte antes de ejecutar. En la misma prueba se vio que el monitor entraba y salía de somnoliento cada 15 s mientras el PERCLOS seguía alto; se cambió la condición de salida para exigir también que el PERCLOS baje.

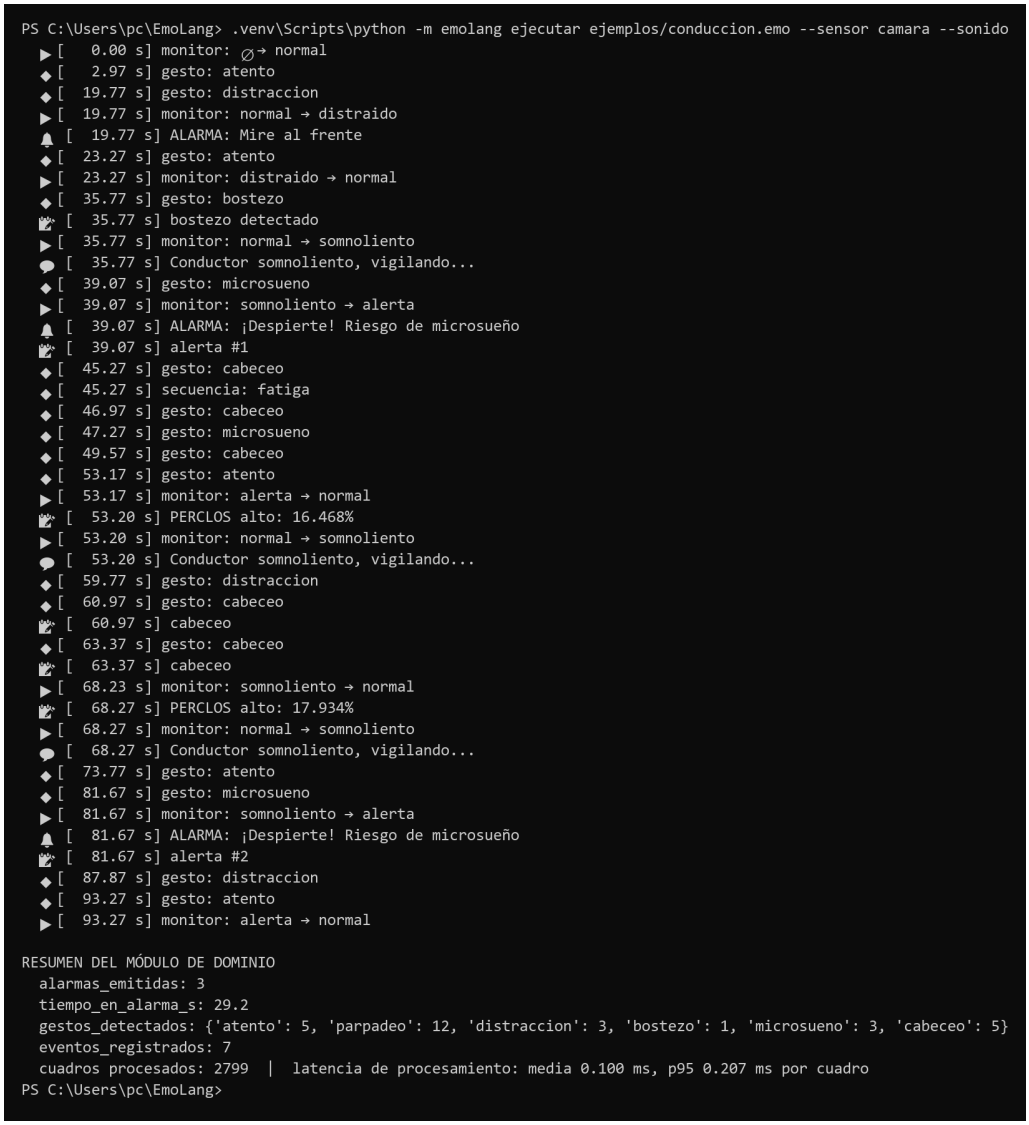


Figura 16. Ejecución en vivo con la cámara (versión del programa anterior a las correcciones descritas).

12. Cambios respecto al anteproyecto

Ningún elemento del anteproyecto se eliminó. Los siguientes cambios y agregados surgieron durante el desarrollo y se someten a revisión:

Tabla 11. Cambios y agregados respecto al anteproyecto.

Elemento	Cambio	Motivo
Cláusula hasta en gestos	Agregada: duración máxima de un gesto.	Distinguir un parpadeo de un microsueño con la misma condición.
Cláusula tolerando	Agregada.	Un cuadro con ruido reiniciaba los gestos sostenidos (sección 11.3).
Instrucción ignorar y nivel nota	Agregados.	Detectar eventos atendidos en unos estados y olvidados en otros (sección 11.4).
Declaración de gestos y secuencias	Con su propia palabra (gesto, secuencia) y no con var.	Un gesto no es un valor que cambie; es un patrón que se compila a un autómata.
mientras	Sólo puede declararse directamente en un estado.	Se evalúa una vez por cuadro; dentro de un bloque de una sola ejecución no tendría sentido.
ms y s	Son palabras reservadas.	Permite escribir 100 ms y 2 s sin ambigüedad; no pueden usarse como nombres.
Conversiones explícitas	Funciones a_porcentaje, a_proporcion, a_ms, a_s.	Hacer visibles las conversiones que no son implícitas.
decimal → int	Prohibida implícitamente.	Evitar pérdida de precisión silenciosa.
Sensores de prueba	Sensor simulado, reproducción de CSV, captura etiquetada y diagnóstico.	Probar y evaluar sin depender de la cámara.

13. Limitaciones y trabajo pendiente

- Falta la evaluación con varios participantes. Los resultados cuantitativos son de datos simulados; las pruebas con cámara las hizo una sola persona, y los umbrales de los ejemplos se ajustaron con sus mediciones.
- Mirar hacia abajo (por ejemplo, al tablero) también reduce ear a 0.18–0.21, cerca del umbral de microsueño. Hay que revisarlo con grabaciones reales.
- La magnitud de pitch es menor de lo esperado (unos 20° entre mirar arriba y abajo). Su signo es correcto, pero falta confirmar si la medición se queda corta o si los movimientos fueron pequeños.
- Con tolerancia, los autómatas de gestos largos crecen a cientos de estados. No afecta el rendimiento, pero las tablas completas son difíciles de leer; los diagramas agrupan los estados intermedios.
- El parpadeo no puede etiquetarse a mano (dura unos 150 ms), así que en la evaluación real sólo se medirá con datos simulados.
- No hay interfaz gráfica: el uso es desde la terminal, con una ventana de video que muestra las métricas, el estado de cada máquina y las alarmas.

El siguiente paso es aplicar el protocolo de evaluación (docs/protocolo_evaluacion.md): grabar con cada participante una sesión de conducción simulada, una normal y una de rehabilitación, etiquetando los gestos con el comando capturar, y medir exactitud, falsos positivos y negativos y

latencia con evaluar. Para no sobreajustar, los umbrales se ajustarán con unos participantes y los resultados se reportarán con otros.

14. Puntos para revisión

El equipo solicita retroalimentación sobre los siguientes puntos:

1. ¿Se aceptan los agregados al lenguaje (hasta, tolerando, ignorar) o deben quedar fuera del alcance del proyecto?
2. ¿Es adecuado escribir los analizadores a mano, o se espera el uso de un generador como ANTLR, PLY o Flex/Bison?
3. ¿La forma de presentar los autómatas (tabla de transiciones más diagrama, con estados agrupados en los gestos largos) cubre lo que pide el objetivo 5?
4. ¿El protocolo de evaluación (3 a 5 participantes, tres sesiones cada uno, etiquetado manual) es suficiente para el objetivo 8?
5. ¿Se mantienen ambos módulos de dominio (conducción y rehabilitación) o conviene concentrarse en uno para la entrega final?
6. ¿Hay algún formato o sección que deba tener el reporte final que no esté en este avance?

Referencias

House, J. W. y Brackmann, D. E. (1985). Facial nerve grading system. *Otolaryngology–Head and Neck Surgery*, 93(2), 146–147.

Knuth, D. E., Morris, J. H. y Pratt, V. R. (1977). Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2), 323–350.

Lugaresi, C. et al. (2019). MediaPipe: A framework for building perception pipelines. *arXiv:1906.08172*.

Mernik, M., Heering, J. y Sloane, A. M. (2005). When and how to develop domain-specific languages. *ACM Computing Surveys*, 37(4), 316–344.

Soukupová, T. y Čech, J. (2016). Real-time eye blink detection using facial landmarks. *21st Computer Vision Winter Workshop*.

Wierwille, W. W., Ellsworth, L. A., Wreggit, S. S., Fairbanks, R. J. y Kirn, C. L. (1994). *Research on vehicle-based driver status/performance monitoring: development, validation, and refinement of algorithms for detection of driver drowsiness*. National Highway Traffic Safety Administration, DOT HS 808 247.

Anexo A. Programa del módulo de conducción

```
// EmoLang. Módulo de transporte: detección de fatiga.
dominio conduccion {

    sensor camara {
        var fps : int = 30;
    }

    // Umbrales configurables por el supervisor de transporte
    var umbral_ojo     : proporcion = 0.20;
    // Valores ajustados con el diagnóstico de cámara (python -m emolang diagnostico)
    var umbral_bostezo : proporcion = 33%;
    var umbral_perclos : proporcion = 15%;
    var cabeceo_grados : decimal    = 10.0;
    var giro_grados    : decimal    = 20.0;
    var alertas        : int        = 0;

    // Gestos: condiciones sostenidas en el tiempo.
    // 'tolerando' perdona interrupciones breves (ruido o cuadros perdidos)
    // sin reiniciar el conteo de la duración.
    gesto parpadeo     = ear < umbral_ojo y mar < umbral_bostezo durante 60 ms hasta 400 ms;
    gesto microsueno   = ear < umbral_ojo y mar < umbral_bostezo durante 1 s tolerando 100 ms;
    gesto bostezo      = mar > umbral_bostezo durante 1.5 s tolerando 200 ms;
    gesto cabeceo      = pitch > cabeceo_grados y ear < 0.27 durante 300 ms tolerando 100 ms;
    gesto distraccion  = yaw > giro_grados o yaw < -giro_grados durante 2 s tolerando 200 ms;
    gesto atento       = pitch < 10.0 y yaw < 20.0 y yaw > -20.0 durante 3 s tolerando 200 ms;

    // Patrón de fatiga: bostezo seguido de cabeceo en menos de 10 s
    secuencia fatiga = [bostezo, cabeceo] en 10 s;

    // Máquina de estados del monitor.
    maquina monitor inicial normal {

        estado normal {
            var parpadeos : int = 0;
            al_entrar { detener_alarma(); }

            cuando parpadeo   { parpadeos = parpadeos + 1; }
            cuando bostezo    { registrar("bostezo detectado"); ir_a somnoliento; }
            cuando microsueno ir_a alerta;
            cuando distraccion ir_a distraido;
            cuando fatiga     ir_a alerta;
            ignorar atento;                      // estar atento ya es lo normal

            mientras perclos > umbral_perclos {
                registrar("PERCLOS alto: " + perclos);
                ir_a somnoliento;
            }
        }

        estado somnoliento {
            al_entrar { mostrar("Conductor somnoliento, vigilando..."); }
            cuando microsueno ir_a alerta;
            cuando fatiga     ir_a alerta;
            cuando distraccion ir_a distraido;   // somnoliento y sin mirar al frente
            cuando cabeceo    { registrar("cabeceo"); }
            ignorar bostezo, atento;             // ya se sabe que está somnoliento
            // solo se relaja cuando pasaron 15 s Y el PERCLOS ya bajó
            mientras tiempo_en_estado() > 15 s y perclos < umbral_perclos {
```

```
                ir_a normal;
            }
        }

        estado distraido {
            al_entrar { alarma("Mire al frente"); }
            cuando atento ir_a normal;
            cuando microsueno ir_a alerta;
            ignorar bostezo, distraccion, fatiga; // la alarma ya está sonando
        }

        estado alerta {
            al_entrar {
                alertas = alertas + 1;
                alarma("¡Despierte! Riesgo de microsueño");
                registrar("alerta #" + alertas);
                si alertas >= 3 {
                    registrar("Se recomienda detener el vehículo");
                }
            }
            cuando atento ir_a normal;
            ignorar bostezo, microsueno, distraccion, fatiga;  // alarma activa hasta estar atento
        }
    }
}
```

Anexo B. Programa del módulo de rehabilitación

```
// EmoLang. Módulo de salud: rehabilitación facial.
// Ejercicio: sonrisa sostenida, 8 repeticiones, con medición de
// simetría (apoyo a seguimiento tipo House-Brackmann).
dominio rehabilitacion {

    sensor camara {
        var fps : int = 30;
    }

    var meta             : int        = 8;
    var umbral_sonrisa   : proporcion = 40%;
    var umbral_reposo    : proporcion = 0.15;
    var simetria_minima  : proporcion = 80%;
    var sostener         : duracion   = 500 ms;

    gesto sonreir = sonrisa > umbral_sonrisa durante sostener tolerando 100 ms;
    gesto reposo  = sonrisa < umbral_reposo  durante 300 ms;

    maquina ejercicio inicial preparacion {

        estado preparacion {
            al_entrar { mostrar("Relaje el rostro para comenzar"); }
            cuando reposo ir_a esperando;
            ignorar sonreir;            // primero debe relajar el rostro
        }

        estado esperando {
            var recordado : bool = falso;
            cuando sonreir ir_a contraccion;
            ignorar reposo;             // ya está en reposo
            mientras tiempo_en_estado() > 10 s y no recordado {
                mostrar("Recuerde: sonría ampliamente y sostenga");
                recordado = verdadero;
            }
        }

        estado contraccion {
            ignorar sonreir;            // la sonrisa sostenida ya se contó
            al_entrar {
                contar_repeticion();
                mostrar("Repetición", repeticiones(), "de", meta, "- simetría", simetria);
                si simetria < simetria_minima {
                    registrar("simetría baja en repetición " + repeticiones());
                }
            }
            cuando reposo {
                si repeticiones() >= meta {
                    ir_a terminado;
                } sino {
                    ir_a esperando;
                }
            }
        }

        estado terminado {
            al_entrar {
                mostrar("¡Ejercicio completo!");
                repetir 3 veces { registrar("fin de sesión"); }
```

```
            }
        }
    }
}
```

Anexo C. Cómo reproducir los resultados

```
python -m unittest discover -s tests -t .
python -m emolang compilar  ejemplos/conduccion.emo
python -m emolang ejecutar  ejemplos/conduccion.emo
python -m emolang evaluar   ejemplos/conduccion.emo --repeticiones 20 --ruido 3
python -m emolang evaluar   reporte/conduccion_version_inicial.emo --repeticiones 20 --ruido 3

.venv\Scripts\python -m emolang diagnostico
.venv\Scripts\python -m emolang ejecutar ejemplos/conduccion.emo --sensor camara --sonido
.venv\Scripts\python -m emolang capturar grabaciones/p1_conduccion.csv
```